



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático 2 - Sistemas Distribuídos

HENRIQUE ALVES CAMPOS [EF04231]

HENRIQUE DE SOUZA CAMPOS [EF04673]

HERON FILLIPE SILVEIRA SANTOS [EF04211]

Sumário

Sumário.....	2
1. Introdução.....	3
2. Modelo Físico.....	3
3. Modelo de Arquitetura.....	4
3.1. Paradigma de Comunicação.....	4
3.2. Modelo Arquitetural.....	4
3.3. Mapeamento de Casos de Uso.....	4
3.4. Estrutura em Camadas Físicas.....	5
3.5. Estrutura em Camadas Lógicas.....	6
4. Modelos Fundamentais.....	6
4.1. Modelos de Interação.....	6
4.2. Comunicação Síncrona ou Assíncrona.....	7
4.3. Modelo de Falhas.....	7
4.4. Modelo de Segurança.....	8
5. Conclusão.....	8
Referências.....	9

1. Introdução

Nesta segunda etapa do trabalho prático, dá-se continuidade ao desenvolvimento do sistema distribuído Papel-it, previamente especificado na Parte 1. Enquanto a etapa anterior teve como foco a definição dos requisitos funcionais e a modelagem por meio de casos de uso, esta fase concentra-se na modelagem conceitual e arquitetural do sistema.

O objetivo desta etapa é analisar e definir os principais aspectos relacionados ao modelo físico adotado, à arquitetura do sistema e aos modelos fundamentais de sistemas distribuídos, incluindo interação, falhas e segurança. Essas definições são essenciais para orientar a implementação do sistema, garantindo coerência entre os requisitos previamente levantados e as decisões de projeto adotadas.

2. Modelo Físico

Entre os três modelos físicos comumente adotados em sistemas distribuídos, o workstation model, o processor pool model e o hybrid model, o sistema Papel-it adota o workstation model como o mais adequado à sua proposta.

Nesse modelo, cada usuário dispõe de sua própria máquina cliente, com capacidade de processamento local, que se conecta a um servidor central responsável pelo estado compartilhado da aplicação. Essa configuração reflete diretamente a forma como o Papel-it é concebido: múltiplos usuários acessando o mural simultaneamente a partir de suas próprias estações de trabalho.

O processor pool model seria mais adequado para cenários nos quais os usuários não possuem máquinas dedicadas e compartilham um conjunto dinâmico de processadores, o que não corresponde ao perfil do sistema proposto. Já o hybrid model combina características dos dois anteriores, sendo mais indicado para sistemas de maior escala e complexidade, o que também não se justifica para o escopo do Papel-it.

A escolha pelo workstation model está alinhada com o público-alvo do sistema, composto por usuários em ambientes de pequeno a médio porte, como laboratórios ou redes locais acadêmicas. Essa abordagem oferece um bom equilíbrio entre simplicidade de implementação, consistência do estado global e adequação ao contexto de uso previsto.

3. Modelo de Arquitetura

3.1 Paradigma de Comunicação

O sistema Papel-it utiliza o paradigma de comunicação baseado em troca de mensagens, no qual as entidades do sistema interagem por meio do envio e recebimento de mensagens estruturadas.

A comunicação é realizada utilizando o protocolo TCP, garantindo confiabilidade na entrega das mensagens entre clientes e servidor. Os clientes enviam requisições contendo operações a serem executadas como criação, edição e remoção de post-its, enquanto o servidor processa essas requisições e responde com mensagens contendo o resultado da operação ou atualizações do estado global.

Esse modelo permite desacoplamento entre as entidades e é amplamente utilizado em sistemas distribuídos devido à sua flexibilidade e simplicidade de implementação.

3.2 Modelo Arquitetural

O modelo arquitetural adotado pelo sistema é o modelo cliente-servidor, no qual o servidor central é responsável pela gestão do estado global da aplicação, enquanto os clientes atuam como interfaces de interação com o usuário.

Nesse modelo, o servidor desempenha o papel de coordenador, sendo responsável por validar operações, garantir a consistência dos dados e distribuir atualizações para os clientes conectados. Os clientes, por sua vez, são responsáveis por capturar as ações do usuário e enviá-las ao servidor na forma de requisições.

Essa abordagem é adequada para o sistema proposto, pois facilita o controle de concorrência e a sincronização do estado compartilhado entre múltiplos usuários.

3.3 Mapeamento de Casos de Uso

Os casos de uso definidos na Parte 1 do trabalho são mapeados para as entidades arquitetônicas cliente e servidor da seguinte forma:

UC01 — Conectar ao Sistema: o cliente envia a requisição de conexão contendo a identificação do usuário. O servidor registra o usuário e retorna o estado atual da

matriz ao cliente, que o exibe ao usuário.

UC02 — Visualizar Mural: o cliente apresenta localmente o estado atual da matriz, mantendo a visualização sincronizada com as atualizações recebidas do servidor.

UC03 — Criar Papel-it: o cliente captura a posição e o conteúdo informados pelo usuário e envia a requisição ao servidor. O servidor valida se a posição está disponível, registra o novo papel-it associando-o ao usuário e ao horário da operação, atualiza o estado global e notifica todos os clientes conectados.

UC04 — Editar Papel-it: o cliente envia ao servidor a requisição de edição contendo a posição e o novo conteúdo. O servidor valida a existência do papel-it, aplica a alteração, registra o horário da modificação e notifica todos os clientes conectados.

UC05 — Remover Papel-it: o cliente envia ao servidor a requisição de remoção indicando a posição do papel-it. O servidor valida a existência do papel-it, remove-o da matriz, atualiza o estado global e notifica todos os clientes conectados.

UC06 — Receber Atualizações: o servidor detecta qualquer alteração no estado do mural e envia notificações a todos os clientes conectados. Cada cliente recebe a atualização e aplica a alteração localmente, mantendo a visualização consistente com o estado global.

Em todos os casos de uso, a lógica de negócio e a validação das operações são responsabilidade exclusiva do servidor, enquanto o cliente atua como interface entre o usuário e o sistema.

3.4 Estrutura em Camadas Físicas

Do ponto de vista físico, o sistema pode ser estruturado em duas camadas principais:

- **Camada Cliente:** composta pelas aplicações cliente responsáveis pela interface com o usuário e envio de requisições.
- **Camada Servidor:** responsável pelo processamento das requisições, gerenciamento do estado global e controle de comunicação entre os clientes.

Essa divisão reflete a separação entre os componentes distribuídos do sistema e facilita a compreensão da organização física da aplicação.

3.5 Estrutura em Camadas Lógicas

Do ponto de vista lógico, o sistema pode ser estruturado em múltiplas camadas, cada uma com responsabilidades bem definidas:

- **Camada de Apresentação:** responsável pela interface com o usuário, permitindo a interação com o sistema por exemplo a interface gráfica.
- **Camada de Comunicação:** responsável pelo envio e recebimento de mensagens entre cliente e servidor.
- **Camada de Lógica de Negócio:** responsável pela validação das operações, controle de concorrência e aplicação das regras do sistema.
- **Camada de Dados:** responsável pela representação e armazenamento do estado global do sistema, como a matriz de post-its.

Essa separação em camadas promove maior organização, modularidade e facilidade de manutenção do sistema.

4. Modelos Fundamentais

4.1 Modelo de Interação

No sistema Papel-it, os fatores de desempenho da rede possuem impactos distintos na interação entre cliente e servidor.

A latência exerce influência moderada, uma vez que o sistema possui caráter colaborativo e espera-se que as atualizações realizadas por um usuário sejam refletidas nos demais clientes em tempo próximo ao real. Latências elevadas podem causar atrasos na visualização das alterações, prejudicando a experiência do usuário.

A taxa de transmissão de dados não representa um fator crítico, pois o volume de dados trafegado é relativamente baixo, consistindo principalmente em mensagens contendo texto, coordenadas e metadados simples.

O jitter, por sua vez, possui impacto reduzido no sistema, já que pequenas variações no tempo de entrega das mensagens não comprometem significativamente a consistência ou a funcionalidade da aplicação.

4.2 Comunicação Síncrona ou Assíncrona

O sistema Papel-it adota um modelo de comunicação predominantemente assíncrono, no qual o servidor envia notificações de atualização do mural a todos os clientes conectados de forma independente, sem que estes precisem realizar novas requisições para receber as mudanças realizadas por outros usuários.

No entanto, para operações iniciadas diretamente pelo usuário, como criação, edição e remoção de papel-its, o comportamento se aproxima do modelo síncrono. Nesses casos, o cliente aguarda a confirmação do servidor antes de considerar a operação concluída, o que está alinhado com os fluxos principais descritos nos casos de uso UC03, UC04 e UC05 na Parte 1 do trabalho.

Essa combinação é adequada ao sistema proposto. A confirmação síncrona das operações garante que o usuário tenha ciência imediata do sucesso ou falha de suas ações, enquanto o recebimento assíncrono de atualizações permite que o mural seja mantido sincronizado entre todos os clientes sem necessidade de consultas periódicas ao servidor.

4.3 Modelo de Falhas

Dentre as classes de falhas estudadas, destaca-se a possibilidade de ocorrência de falhas por omissão no sistema Papel-it.

Esse tipo de falha pode ocorrer, por exemplo, quando o servidor deixa de responder a uma requisição de um cliente devido a problemas de rede, sobrecarga ou interrupção inesperada. Nesse cenário, o cliente não recebe confirmação da operação realizada, podendo gerar inconsistência na percepção do estado do sistema.

A detecção dessa falha pode ser realizada por meio de mecanismos de timeout, nos quais o cliente identifica a ausência de resposta dentro de um intervalo de tempo pré-definido.

Como forma de tratamento, o sistema pode permitir o reenvio da requisição pelo cliente ou exibir uma mensagem de erro informando a indisponibilidade temporária do servidor. Em implementações mais robustas, poderiam ser adotadas estratégias adicionais, como replicação de servidores ou mecanismos de retry automático.

4.4 Modelo de Segurança

Proteção de Processos

A proteção aos processos possui relevância moderada no contexto do sistema Papel-it. Como múltiplos usuários interagem sobre um ambiente compartilhado, a ausência de mecanismos de controle de identidade e permissões permite que qualquer usuário conectado edite ou remova papel-its criados por outros, comprometendo a integridade das informações do mural.

Em um cenário acadêmico e de uso controlado, esse risco é reduzido, uma vez que se pressupõe boa-fé entre os participantes. Ainda assim, seria recomendável, em um ambiente real, a adoção de controle de autoria, garantindo que operações de edição e remoção sejam restritas ao usuário responsável pela criação do papel-it.

O sistema proposto implementa de forma básica a associação de cada papel-it ao seu autor, o que representa um primeiro nível de rastreabilidade, ainda que sem restrição formal de acesso.

Proteção ao Canal de Comunicação

A proteção ao canal de comunicação não é estritamente necessária para o escopo acadêmico do sistema, especialmente quando utilizado em redes locais controladas. Nesse ambiente, o risco de interceptação de mensagens por terceiros é baixo.

No entanto, em um cenário de uso real e aberto, a ausência de proteção expõe as mensagens trafegadas entre clientes e servidor, permitindo que um agente externo intercepte o conteúdo dos papel-its, identifique os usuários conectados ou até injete mensagens falsas no sistema, corrompendo o estado do mural. A adoção de um protocolo seguro como TLS mitigaria esses riscos, garantindo confidencialidade e integridade na comunicação.

5. Conclusão

A modelagem do sistema distribuído Papel-it, apresentada nesta etapa, permitiu consolidar as decisões arquiteturais e estruturais necessárias para sua implementação. A definição do modelo físico centralizado, aliada ao uso da arquitetura cliente-servidor e do paradigma de comunicação baseado em troca de mensagens, mostrou-se adequada para atender aos requisitos levantados na etapa anterior.

Além disso, a análise dos modelos fundamentais interação, falhas e segurança possibilitou a identificação de aspectos relevantes relacionados ao desempenho, à confiabilidade e à proteção do sistema, contribuindo para uma visão mais completa do seu funcionamento em um ambiente distribuído.

De modo geral, as escolhas realizadas priorizam a simplicidade de implementação e a consistência do estado compartilhado, características importantes para o escopo do sistema proposto. Como trabalhos futuros, podem ser exploradas melhorias como maior tolerância a falhas, mecanismos mais robustos de segurança e possíveis evoluções para arquiteturas descentralizadas, visando maior escalabilidade.

Referências

[1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2. ed. São Paulo: Pearson, 2007.

[2] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. Distributed Systems: Concepts and Design. 5. ed. Boston: Addison-Wesley, 2011.

[3] PYTHON SOFTWARE FOUNDATION. socket — Low-level networking interface. Disponível em: <https://docs.python.org/3/library/socket.html>. Acesso em: 2026.

[4] PYTHON SOFTWARE FOUNDATION. threading — Thread-based parallelism. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 2026.

[5] PYTHON SOFTWARE FOUNDATION. tkinter — Python interface to Tcl/Tk. Disponível em: <https://docs.python.org/3/library/tkinter.html>. Acesso em: 2026.

[6] GUEDES, Gilleanes T. A. UML 2: Guia Prático. São Paulo: Novatec, 2011.